TECHNICAL DESIGN DOCUMENT

Theta Chat Bot

Architecture, API, and AI Guardrails Strategy

Field	Value
Project Name	Theta Chat Bot
Prepared For	Theta Technolabs
Author	Shramika Chavda
Document Version	2.0 (Condensed)
Document Status	Internal Use Only
Date	July 2026

DOCUMENT INDEX

Table of Contents

CHAPTER 01 — EXECUTIVE SUMMARY

1. Executive Summary

1.1 Introduction

The **Theta Chat Bot** is a modular, AI-powered virtual assistant for the **Theta Technolabs website**. It provides instant, accurate responses based exclusively on the company's publicly available content — services, portfolios, case studies, careers, and blogs — using a **Retrieval-Augmented Generation (RAG)** architecture. User queries are matched against indexed website content before a Large Language Model (LLM) generates a grounded response, keeping the system model-agnostic and free to use high-performance or cost-effective providers as needed.

Two deployment models support different business needs: a **Lightweight Architecture** (async API + vector database + LLM) for fast, low-overhead deployment, and an **Enterprise Architecture** (async API + relational & vector database + admin dashboard) for advanced analytics and operational monitoring. The chatbot is embedded as a floating widget on the existing website and fully containerized for on-premise or cloud deployment.

1.2 Why This Solution & Project Goals

Website visitors expect conversational interfaces that interpret natural language and deliver concise answers, rather than manually curated FAQs. The chatbot dynamically retrieves relevant content and generates human-like, grounded responses. Business advantages include reduced support workload, higher engagement, automatic alignment with website updates (no retraining needed), and a scalable foundation for dashboards and analytics.

Core project goals: deliver 24/7 conversational assistance; automate common inquiries for sales/support; increase engagement and conversion by guiding users to relevant pages; ensure high accuracy grounded strictly in approved content; and establish an extensible AI platform.

1.3 Key Features

- **Semantic Search** — understands intent beyond exact keyword matching.
- **Context-Aware Answering** — retrieves relevant chunks before generating a response.
- **Source Citations** — direct page references for transparent verification.
- **Session-Based Memory** — maintains context across follow-up questions.
- **Fallback Handling** — politely declines off-topic queries and redirects to contact channels.
- **Embeddable UI Widget** — floating chat interface across all website pages.

1.4 Scope

In Scope	Out of Scope
Company information, AI/Mobile/Web Development, IoT, Blockchain, Cloud, UI/UX, Case Studies, Blogs, Careers, Contact, Technologies, FAQs	Personal opinions, medical/legal/financial advice, current news, general internet search, information unrelated to Theta Technolabs

1.5 High-Level Solution

For every query, the chatbot executes four steps: **(1) User Query Processing** — validates natural language input; **(2) Semantic Knowledge Retrieval** — searches the vector database for relevant chunks; **(3) Contextual Prompt Construction** — injects retrieved content and guardrails into a structured prompt; **(4) Grounded Response Delivery** — returns a concise, source-referenced answer.

> EXAMPLE INTERACTION
>
> **Visitor:** "Do you develop AI Agents?"
> **Chatbot:** "Yes. Theta Technolabs develops custom AI agents, conversational AI solutions, LLM applications, and intelligent automation systems tailored to business needs." — with a link to the AI Development Services page.

2. Solution Architecture & RAG Approach

2.1 Overview

The chatbot dynamically retrieves relevant website content and supplies it as context to the LLM, rather than storing predefined Q&A pairs or training a custom model. Five modular stages make up the architecture: **Knowledge Acquisition** (content collected and prepared), **Knowledge Storage** (indexed in a vector database), **Query Processing** (user questions analyzed and matched), **Response Generation** (context + query sent to the LLM), and **Response Delivery** (answer returned via the widget). Separating knowledge storage from language generation lets content update independently of the model.

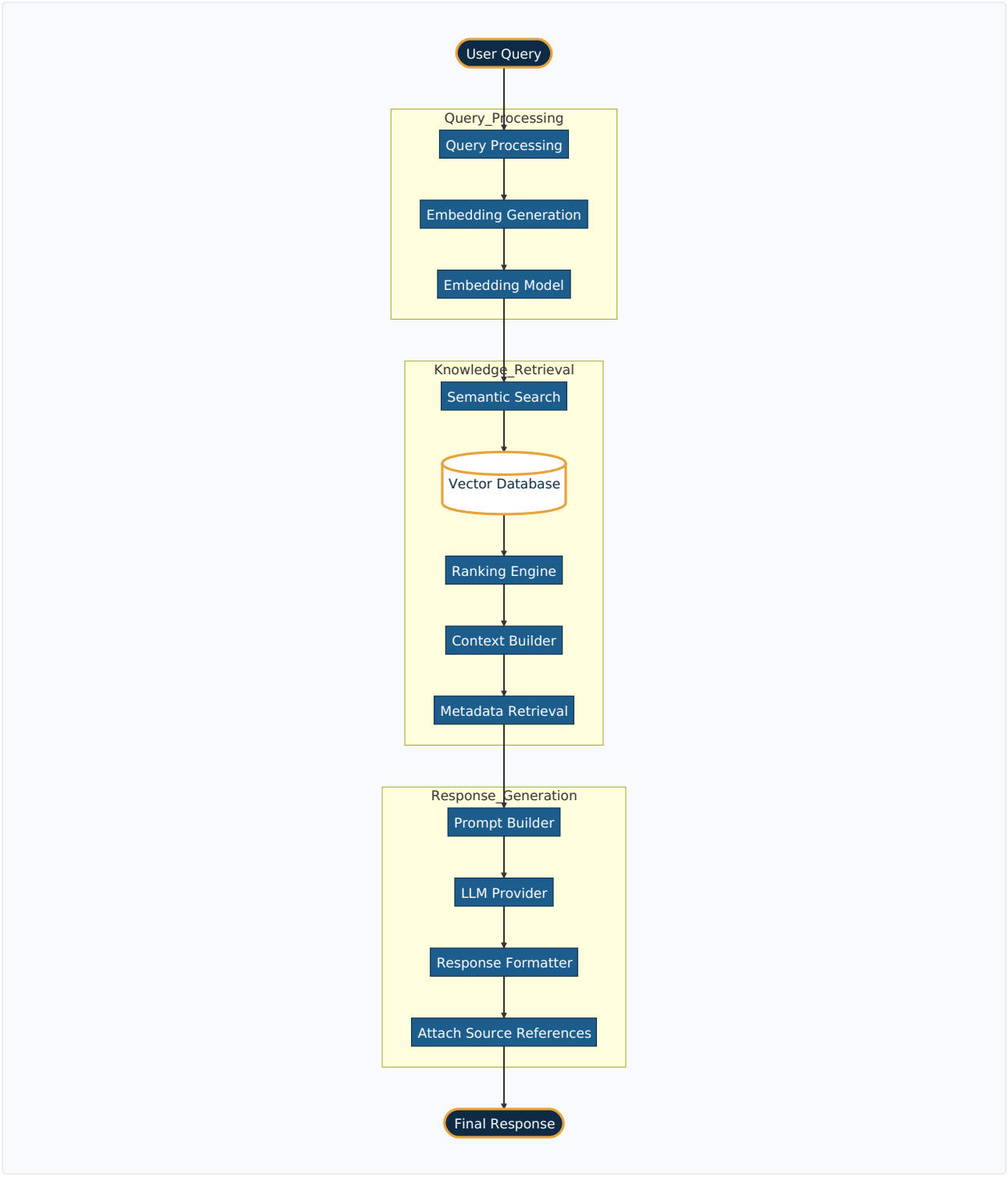
2.2 Why Retrieval-Augmented Generation?

Approach	Limitation
Rule-Based Chatbot	Difficult to maintain, poor scalability
FAQ Matching	Cannot answer complex or unseen questions
Fine-Tuned LLM	Expensive, requires retraining on every content change
RAG (Selected)	Requires a retrieval pipeline and vector database — accepted trade-off for accuracy and maintainability

WHY RAG?

The website is a single source of truth that evolves continuously; fine-tuning on every change would add unnecessary cost and operational overhead. RAG separates knowledge from generation — content is re-indexed as it changes, with no LLM retraining — improving accuracy, maintainability, and provider flexibility.

2.3 System Workflow



The pipeline runs in three phases: **Query Processing** converts the input into embeddings; **Knowledge Retrieval** searches the vector database by semantic similarity; **Response Generation** combines retrieved knowledge with the query to produce a grounded answer via the LLM.

2.4 Key Design Decisions

Decision	Rationale
Website as primary knowledge base	Keeps responses aligned with official company information
RAG instead of fine-tuning	Simplifies updates, reduces operational cost
FastAPI backend	High-performance async handling, strong AI ecosystem
Gemini as initial LLM	Cost-effective with strong reasoning for MVP
Modular architecture	LLM, vector database, or frontend replaceable independently
Docker deployment	Simplifies deployment, portability, on-premise hosting
Optional PostgreSQL dashboard	Enables analytics without affecting core chatbot function

3. Knowledge Management Pipeline

3.1 Content Acquisition & Cleaning

The chatbot is grounded exclusively in publicly available Theta Technolabs website content: Home, About, Services, Technologies, Industry Solutions, Portfolio/Case Studies, Blogs, Careers, and Contact/FAQ pages. Privacy policies, cookie notices, and legal disclaimers are excluded from indexing.

Before indexing, content passes through a preprocessing pipeline: HTML parsing and structured text extraction; removal of navigation, headers/footers, cookie banners, and script/style elements; de-duplication and whitespace normalization; preservation of headings and document hierarchy; metadata extraction (page title, URL, section); and validation to discard empty or low-quality content.

3.2 Content Chunking

Webpages are split using a **Sliding Window Overlap Chunking Strategy** — a **500-token baseline** with **50–100 token overlap**, adjustable at runtime by document type and density. The strategy preserves paragraph/sentence boundaries and keeps headings attached to their body text, with URL, page title, section, and chunk index stored as metadata on every vector record. This improves retrieval precision by returning only the relevant section rather than an entire page.

3.3 Embedding Generation

Each chunk is converted into a numerical vector embedding that captures semantic meaning rather than exact wording — so "Do you build AI chatbots?" correctly matches content describing "intelligent conversational AI solutions" despite different phrasing.

3.4 Knowledge Storage

Requirement Scope	Recommended Architecture	Rationale
Knowledge retrieval only	**Qdrant** (dedicated vector DB)	Lightweight, purpose-built for high-speed semantic search with minimal infrastructure overhead. Limitation: no relational storage — a separate DB is needed for logs/analytics.
Retrieval + logging, analytics, dashboard	**PostgreSQL + pgvector** (unified DB)	Stores relational and vector data in one system, enabling analytics/reporting and enterprise dashboards without a second database. Trade-off: higher resource demands and more admin overhead than Qdrant alone.

3.5 Knowledge Synchronization

Since content changes only when pages are published or updated, scheduled re-crawling would waste compute reprocessing unchanged data. The chatbot instead uses **Event-Driven Incremental Synchronization**: whenever content is created, modified, or deleted, only the affected pages are re-cleaned, re-chunked, re-embedded, and updated in the vector database — reducing processing time and embedding costs while keeping the knowledge base current.

Strategy	Trade-off
Manual Refresh	Simple but time-consuming, depends on manual action
Scheduled Synchronization	Automated but reprocesses unchanged content unnecessarily
Event-Driven Incremental (Recommended)	Efficient and fast; requires CMS/publishing-workflow integration

3.6 Metadata & Key Design Decisions

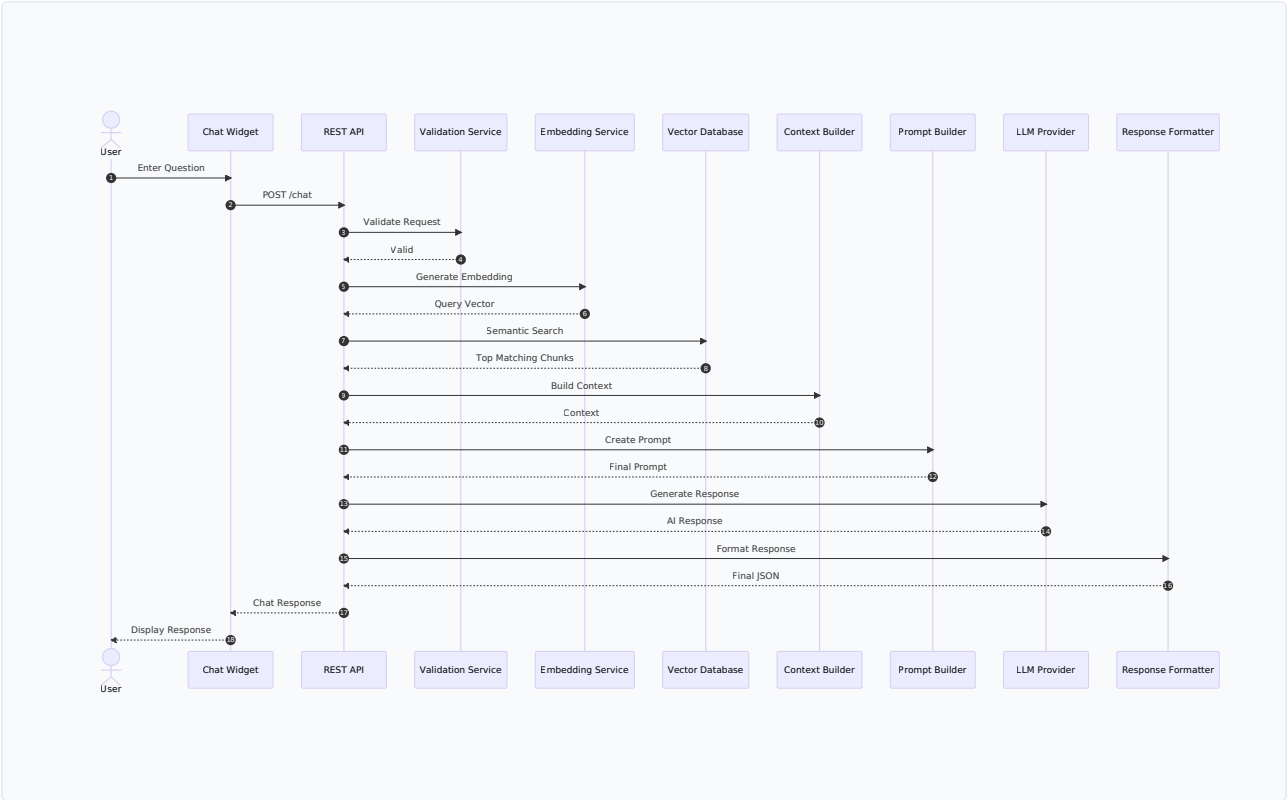
Every chunk stores: Chunk ID, Page URL, Page Title, Section Heading, Content Category, Last Indexed Date, Version, and Vector ID — enabling source citations, versioning, analytics, and auditing.

Decision	Justification
Semantic chunking with overlap	Improves retrieval precision and contextual relevance
Event-driven sync	Updates only modified content, reducing overhead
Qdrant for lightweight / PostgreSQL+pgvector for enterprise	Matches infrastructure to actual analytics and dashboard needs

4. Retrieval & Response Generation

4.1 Overview & Request Sequence

This pipeline transforms a user's question into a grounded, accurate response. Only retrieved knowledge-base content is supplied to the LLM — the pipeline covers query reception, validation, embedding, semantic retrieval, context construction, prompt engineering, generation, response validation, and delivery.



4.2 Query Reception & Embedding

The frontend sends the query via REST to FastAPI, which validates it (empty-message checks, max length, unsupported characters, security filtering, rate limiting) before converting it into a semantic vector using the embedding model — capturing meaning and intent rather than literal keywords.

4.3 Semantic Retrieval

Parameter	Value
Similarity Metric	Cosine Similarity
Retrieved Chunks (Top-K)	Top 3 to 5
Minimum Similarity Threshold	0.60 (strict cutoff)

RETRIEVAL PARAMETER RATIONALE

Top-K (3–5): more chunks inflates cost and dilutes LLM focus ("lost in the middle"); five balances context against noise. **Threshold (0.60):** matches below this are mathematically distant from the knowledge base and are treated as unsupported — the first line of defense against hallucination and out-of-scope queries.

4.4 Prompt Engineering & Context Construction

The backend assembles a structured context (retrieved content, source page info, user question, session history if enabled, system instructions) into a prompt ordered as: **1)** System Instructions, **2)** Retrieved Context, **3)** Conversation Context (optional), **4)** User Question. The system prompt instructs the LLM to answer only from provided content, avoid unsupported claims or speculation, state clearly when information is unavailable, stay concise and professional, and prefer Theta's official terminology.

4.5 Conversation Memory

Session-Based Conversation Memory is the recommended approach — the chatbot remembers prior messages for the active browser session only, discarding history when the session expires. This supports follow-up questions and better contextual understanding at the cost of slightly higher token usage, compared to a simpler stateless mode that cannot handle follow-ups but uses fewer tokens and no session storage.

4.6 Response Validation & Delivery

Before returning an answer, the backend evaluates retrieval confidence, context relevance, and completeness (thresholds detailed in Section 9.5):

Confidence Level	System Behavior
High	Return the answer normally
Medium	Return the answer with a related page reference
Low	State that sufficient information was not found; recommend contacting Theta Technolabs
Out of Scope	Explain the chatbot's scope and redirect to Contact/Inquiry page

The validated response is returned to the frontend as JSON containing the answer, optional source reference, feedback controls (future), and a contact/inquiry action when required.

4.7 Key Design Decisions

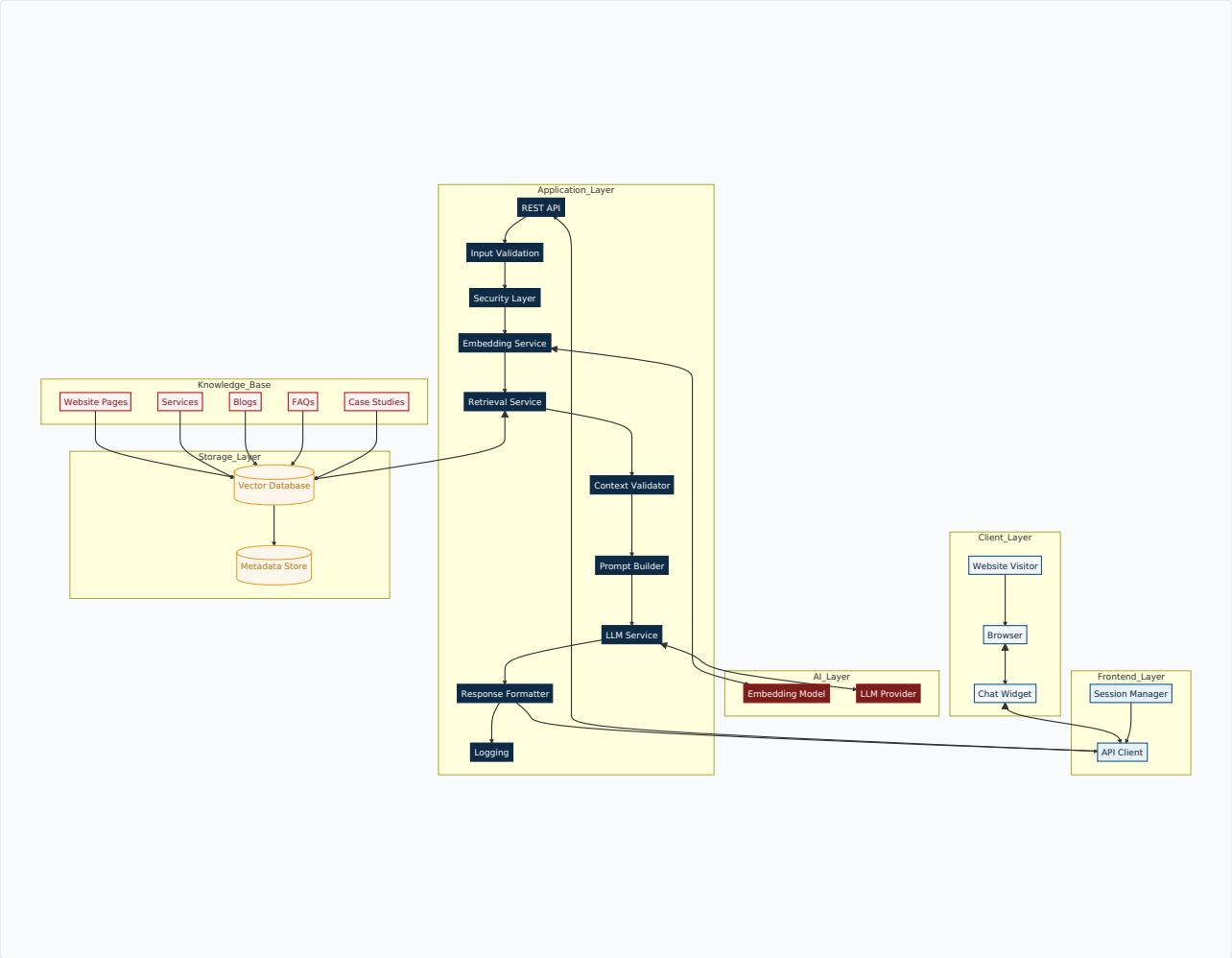
Decision	Justification
Semantic search over keyword search	Better natural-language understanding
Session-based memory	Supports follow-ups without permanent storage
Confidence-based validation	Prevents unsupported or misleading answers
Provider-independent LLM layer	Enables migration between Gemini and OpenAI with minimal effort

5. System Architecture

5.1 Architecture Layers

The system separates concerns into six layers, each independently upgradeable: **Presentation** (embedded JS widget on the existing Webflow site — captures input, displays responses/sources/fallbacks, manages session IDs); **API** (FastAPI REST endpoints — validates requests, routes to the AI pipeline, returns structured JSON); **Business Logic** (orchestrates validation, session management, prompting, error handling, logging); **AI & Retrieval** (embedding, semantic search, context construction, prompt engineering, confidence validation); **Data** (knowledge base and optional analytics store); and **Infrastructure** (Docker containers for portability and on-premise hosting).

5.2 Component Communication



Components communicate in a strictly sequential flow: the widget passes queries to FastAPI, which retrieves semantically similar vectors, constructs a prompt, invokes the LLM, validates the response, and returns the answer with source citations.

5.3 LLM Abstraction Layer

The AI layer is provider-independent. **Primary: Google Gemini** — lower operational cost, strong reasoning, large context window, suited to MVP deployment. **Future option: OpenAI**, swappable without architectural change if testing shows a need for higher reasoning quality. A common interface abstracts the provider:

LLM ABSTRACTION INTERFACE (PSEUDOCODE)

```
class BaseLLMProvider(ABC):
  @abstractmethod
  async def generate_response(self, prompt, context) -> str:
    """Generates a grounded response using the provided context."""
```

5.4 Data & Infrastructure Layer

Architecture A (Lightweight): Qdrant only — stores embeddings, performs similarity search, returns relevant content. Recommended when only chatbot functionality is required.
Architecture B (Enterprise): PostgreSQL + pgvector — stores embeddings plus conversation logs, feedback, API usage, token consumption, dashboard metrics, and configuration. Recommended when an admin dashboard and reporting are required.

The entire solution is containerized with Docker for consistent environments across development, testing, and production. Containers may include the FastAPI app, Nginx reverse proxy, Qdrant or PostgreSQL+pgvector, and pgAdmin (enterprise). Deployment defaults to an on-premise Docker environment for infrastructure control, data management, and lower long-term cost, with the same architecture portable to the cloud if scalability demands increase.

5.5 Scalability & Key Design Decisions

Possible future extensions include an admin dashboard, CRM integration, multilingual support, a voice-based chatbot, multi-website knowledge bases, document upload support, hybrid (semantic + keyword) search, response caching/Redis, and multiple LLM providers — all addable without major redesign due to the layered, modular architecture.

Decision	Justification
Modular layered architecture	Simplifies maintenance and future enhancement
Existing website integration	Eliminates the need to rebuild the frontend
On-premise Docker deployment	Infrastructure control with lower long-term operational cost
Dual database strategy	Matches lightweight or enterprise deployments to actual requirements

6. Technology Stack & Design Rationale

6.1 Selection Criteria

The stack was chosen against four criteria: **Cost** (minimize development, deployment, and operational expense), **Control** (retain ownership of application data and infrastructure), **Development Velocity** (rapid development, testing, and deployment), and **Scalability** (support future multilingual, analytics, and traffic growth without major redesign). The architecture is organized into four independent layers — Frontend, Backend, AI & Knowledge Retrieval, and Infrastructure — reusing the existing Webflow site and keeping infrastructure lightweight and cost-effective.

6.2 Technology Stack Overview

Layer	Technology	Role
Chat Interface	Custom JavaScript Widget	UI, communicates with backend via REST — lightweight, framework-agnostic
Language / Backend	Python + FastAPI	Async REST APIs, AI orchestration, business logic; rich AI ecosystem
AI Model	Google Gemini / OpenAI	Natural language response generation; swappable with minimal code change
Embedding Model	Sentence Transformers	Local vector generation — no per-token API cost, works offline
Vector Store	Qdrant or PostgreSQL + pgvector	Semantic search for RAG (see 3.4 for selection logic)
Knowledge Base	Website pages + Markdown docs	Extendable source content for retrieval
Containerization	Docker & Docker Compose	Consistent environments dev → staging → production
Reverse Proxy	Nginx	HTTPS termination, routing, future load balancing
Hosting	Linux Server (On-Premise)	Secure, controlled, cost-effective within company infrastructure

6.3 Architecture Decisions

ADR — Backend Framework: FastAPI

ALTERNATIVES

Flask (sync by default), Django (too much overhead), Node.js (thinner AI ecosystem), Java/.NET (slower AI iteration)

RATIONALE

The workload is I/O-bound; FastAPI's async handling supports many concurrent chats, integrates natively with Python's AI ecosystem, and auto-generates API docs.

IMPACT

Positive: high performance, one language across ML and API. Trade-off: requires team familiarity with async Python.

ADR — Primary LLM: Google Gemini

ALTERNATIVES

OpenAI (fallback for complex reasoning), self-hosted open-source models (high operational overhead)

RATIONALE

Gemini balances cost, context window, and reasoning quality for RAG; keeping the provider as a config value avoids lock-in, with a data-driven fallback to OpenAI when Gemini fails the quality bar.

IMPACT

Positive: cost-effective, easy to swap providers. Trade-off: maintaining multiple LLM API integrations.

ADR — Local Embeddings: Sentence Transformers

ALTERNATIVES

OpenAI/Gemini embedding APIs (recurring per-token cost)

RATIONALE

Local generation eliminates per-token cost, works offline, keeps data in-house, and guarantees query/document vectors share the same semantic space.

IMPACT

Positive: zero embedding API cost, strong data privacy. Trade-off: consumes local compute.

ADR — Deployment: On-Premise, With a Documented Exit Ramp

ALTERNATIVES

Managed cloud services (elastic scale, ongoing recurring cost)

RATIONALE

On-premise avoids recurring cloud bills for a steady, non-spiky workload; server management is a one-time setup cost. The same Docker architecture can shift to cloud later without redesign.

IMPACT

Positive: lower long-term cost, maximum data control. Trade-off: requires internal server management.

Vector storage (Qdrant vs. PostgreSQL+pgvector) follows the same fork described in Section 3.4, and applies consistently to hosting and monitoring choices. Every service (FastAPI, vector store, Nginx) ships as a container defined once in `docker-compose.yml`, giving identical dev/staging/production environments, one-command startup, and simple rollback by pinning image tags.

6.4 APIs Used

API	Type	Purpose
/api/chat	REST (FastAPI)	Only endpoint the widget calls — accepts a message, returns answer + sources
Gemini API	External	Primary LLM — generates the response from the assembled prompt
OpenAI API (optional)	External	Fallback LLM, used case-by-case when Gemini fails the quality bar
Qdrant REST API	Internal (self-hosted)	Vector storage and similarity search — called only by the backend

6.5 Frontend Integration Approach

Since the website is built on Webflow, the chatbot is delivered as a lightweight embedded JavaScript widget — a single `<script>` tag, with no changes to the existing structure or design. This preserves the current content-management workflow, allows independent styling of the chatbot, and permits future updates without touching the website itself.

7. API Design & Communication Flow

7.1 Overview

The chatbot widget communicates with FastAPI over secure HTTPS REST endpoints using JSON payloads — lightweight, language-independent, and easy to integrate. FastAPI validates requests, retrieves knowledge, communicates with the LLM, and returns structured responses. Flow: `Widget → REST API → Retrieval Engine → Vector DB → Prompt Builder → Gemini API → JSON Response → Widget`.

7.2 Representative Endpoints

Note: representative examples — actual endpoints and payloads may expand as the backend grows.

PROPOSED EXAMPLE ENDPOINTS

```
POST /api/v1/chat — process user queries, generate chatbot responses
POST /api/v1/feedback — submit helpful/unhelpful feedback
GET  /api/v1/health — health check for monitoring/deployment
POST /api/v1/sync — trigger knowledge base sync (admin)
GET  /api/v1/analytics — retrieve usage/operational stats (admin)
GET  /api/v1/config — retrieve chatbot configuration / feature flags
```

7.3 Request & Response Schema

POST /api/v1/chat request body: `session_id` (string, required — browser session identifier), `message` (string, required — user's question), `timestamp` (ISO 8601, optional).

SAMPLE SUCCESS RESPONSE — 200 OK

```
{
  "status": "success",
  "session_id": "c9a3d9b7-f21a-4b8e",
  "response": "Theta Technolabs provides AI Development, Machine Learning...",
  "source": [{"title": "AI Development Services", "url": "..."}],
  "confidence": "High",
  "processing_time_ms": 842
}
```

Success responses always include status, echoed session ID, the generated answer, optional source array, a confidence classification (High/Medium/Low), and processing time — improving transparency and letting users verify answers on the website.

7.4 Error Handling

SAMPLE ERROR RESPONSE — 404 / 500

```
{
  "status": "error",
  "error_code": "KB_404",
  "message": "Relevant information could not be found within the current knowledge base."
}
```

Error Code	Meaning
INVALID_REQUEST	Invalid or incomplete request payload
SESSION_EXPIRED	Session is no longer valid
KB_404	No relevant knowledge found
LLM_TIMEOUT	Language model request timed out
API_LIMIT	API rate limit exceeded
INTERNAL_ERROR	Unexpected server error

HTTP status codes follow standard conventions: 200 OK, 400 Bad Request, 404 Not Found, 429 Too Many Requests, 500 Internal Server Error, 503 Service Unavailable (LLM/DB temporarily down).

7.5 Session Management & Security

No authentication or PII is required; each visitor receives a temporary session identifier enabling follow-up context, conversation memory, and optional analytics. API protections include HTTPS, input validation/sanitization, max message length, CORS configuration, environment-based API key management, and secure server-side LLM communication — credentials are never exposed to the frontend. All public endpoints use URI-based versioning (e.g. `/api/v1/chat`) to allow future changes without breaking existing integrations.

8. Frontend Integration

8.1 Widget Architecture

The chatbot is embedded into the existing Webflow website via a lightweight JavaScript widget communicating with FastAPI over REST — no AI logic lives in the frontend; all retrieval, prompting, and generation happen on the backend.

Component	Responsibilities
Chat Launcher	Floating action button on every page; opens/closes the widget, preserves session state
Chat Window	Conversation history, input, typing indicators, AI responses, source references, fallback options
API Communication Layer	Sends REST requests, receives JSON, handles network failures, retries (future)

The launcher can sit in the main navigation ("Chat with AI") or as a persistent floating button in the bottom-right corner.

8.2 User Journey & UI States

1. **Discover** Visitor opens the website; the chatbot launcher appears bottom-right.
2. **Open** Clicking the icon shows a welcome message: "Hello! 👋 I'm the Theta AI Assistant. How can I help you today?"
3. **Ask** The visitor enters a question; the frontend sends it to the backend.
4. **Wait** A typing indicator displays while the backend processes the request.
5. **Respond** The AI-generated answer displays, with a related page reference if available.
6. **Continue** The user continues the conversation or closes the chatbot.

The widget supports distinct states — Welcome, Idle, Typing, Loading, Response, Error, No Result, and Out of Scope — each giving clear visual feedback.

8.3 Fallback Experience

LOW CONFIDENCE

"I couldn't find enough verified information to answer your question accurately. You may find more details on our website or contact our team for further assistance." — Actions: View Related Services · Contact Us · Submit Inquiry

OUT OF SCOPE

"I'm designed to answer questions related to Theta Technolabs, including our services, technologies, portfolio, careers, and company information. I can't assist with unrelated topics." — Actions: Visit Services · Contact Sales · Submit Inquiry

8.4 Responsive Design, Accessibility & Key Decisions

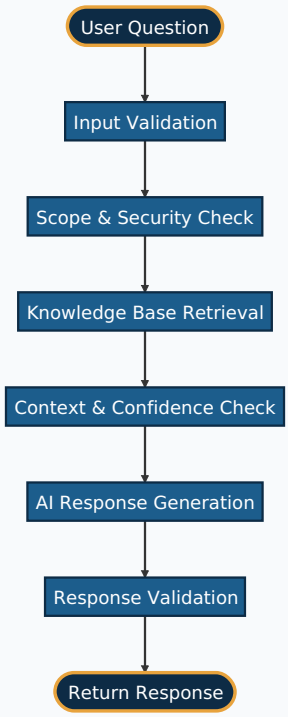
The interface targets a consistent experience across desktop, tablet, and mobile with responsive layout, scrollable history, touch-friendly controls, and an adaptive window size. Accessibility practices include keyboard navigation, screen reader compatibility, sufficient color contrast, readable typography, visible focus indicators, and accessible button labels. Future enhancements under consideration: voice interaction, file upload, multi-language interface, rich response cards, appointment booking, conversation export, and suggested questions on load.

Decision	Justification
Embedded JS widget	Seamless integration without redeveloping the website
Backend-driven AI processing	Keeps the frontend lightweight and secure
Confidence-based fallback	Prevents misleading responses, improves trust

9. AI Guardrails Strategy

9.1 Guardrail Workflow

Every response passes through multiple validation steps before reaching the user, ensuring answers are grounded in the approved knowledge base and preventing hallucinations, prompt injection, and out-of-scope replies.



9.2 Scope & Knowledge Base Guardrails

The chatbot only answers questions about Theta Technolabs — company information, services, technologies, AI solutions, mobile/web development, careers, contact info, blogs, and case studies. Everything else is treated as out-of-scope.

> EXAMPLE
>
> **User:** Who won the FIFA World Cup?
> **Bot:** "I'm designed to answer questions related to Theta Technolabs and its services. For information outside this scope, please refer to an appropriate external source."

Every response must be based on retrieved knowledge-base content; the chatbot never attempts to guess an answer when no relevant content is available.

9.3 Confidence Guardrail

Similarity Score	Action
≥ 0.80	Generate a confident response using the retrieved context
0.60 – 0.79	Generate a response while limiting unsupported details
< 0.60	Treat as unsupported; return a fallback response

This directly maps to the response-validation behavior described in Section 4.6.

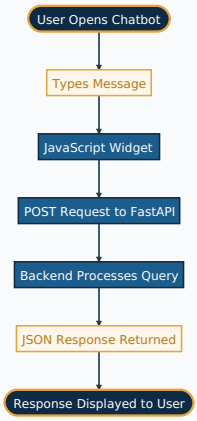
9.4 Prompt & Hallucination Guardrails

> **EXAMPLE SYSTEM PROMPT**
>
> "You are the official Theta Technolabs assistant. Answer only using the provided context. Do not invent information. Do not answer unrelated questions. If the requested information is unavailable, politely state that you cannot find it and recommend visiting the official website."

The model must never fabricate information (pricing, commitments, unlisted services) that is not in the approved knowledge base — a standard fallback message is returned instead. Responses follow a consistent structure: direct answer, concise explanation, and a link to the relevant webpage (target 150–250 words).

9.5 Security & Fallback Strategy

The chatbot safely handles attempts to manipulate its behavior or expose system internals (prompt injection, API keys, system instructions), and politely declines unrelated topics (sports, weather, politics, general coding) with a standard scope clarification.



Benefits: responses stay grounded in approved information; hallucinations and prompt-injection risk are reduced; communication stays consistent and verifiable; and out-of-scope or low-confidence queries get predictable fallback handling.

10. Operational Dashboard & Analytics

10.1 Overview & Architecture

An **optional** Operational Dashboard gives administrators visibility into usage, AI performance, cost, and interactions — recommended for production deployments needing long-term monitoring and business insight. When enabled, PostgreSQL + pgvector is the primary database, storing both embeddings and operational data (conversation metadata, feedback, API requests, token usage, response time, error logs, sync history, dashboard config) in a single platform.

10.2 Dashboard Modules

A. CONVERSATION ANALYTICS

Total conversations, volume trends, average length/response time, active sessions — measures adoption and engagement.

B. KNOWLEDGE ANALYTICS

Most accessed pages/services, FAQs, low-confidence and unanswered queries, out-of-scope requests — flags content gaps.

C. AI USAGE ANALYTICS

API requests, input/output/total tokens, estimated cost, processing time, active LLM provider — controls operational expense.

D. USER FEEDBACK ANALYTICS

Helpful/Not Helpful ratios, trends, comments, most disliked responses — supports continuous quality improvement.

E. SYSTEM HEALTH MONITORING

API availability, error rate, latency, sync status, container health, DB connectivity, storage utilization.

10.3 Database Design & Example KPIs

Table	Purpose
conversations	Session-level conversation information
messages	Individual chatbot and user messages
feedback	User feedback records
api_usage	API request statistics and token usage
sync_history	Knowledge synchronization history
error_logs	Application and API errors

Homepage KPIs: total conversations, messages today, average response time, knowledge coverage (% of queries answered from indexed content), API cost, positive feedback %, failed requests, and last synchronization timestamp.

10.4 Availability & Key Design Decisions

Deployment	Dashboard Included
Lightweight (Qdrant)	Not included
Enterprise (PostgreSQL + pgvector)	Included

Organizations needing only chatbot functionality can deploy without analytics and adopt the enterprise architecture later without changing the overall chatbot design. Future enhancements include real-time/interactive charts, geographic and device reports, CRM integration, lead-generation analytics, exports, email reports, and role-based access control.

11. Security & Bot Prevention

11.1 Security Objectives & Communication Security

Although the chatbot requires no authentication, it follows a **defense-in-depth** approach: protect backend services from unauthorized access, secure all inter-component communication, prevent public API abuse, protect AI provider credentials, maintain knowledge base integrity, and ensure reliable operation. All frontend-to-backend traffic uses HTTPS with automatic HTTP→HTTPS redirection.

11.2 API & Credential Security

Public APIs apply request validation, input sanitization, maximum message length, request timeouts, rate limiting (production), CORS configuration, and versioning. Credentials for Gemini and OpenAI follow strict handling: stored in environment variables, never hardcoded, rotated periodically, permission-restricted where supported, and kept separate per environment (dev/test/prod) — all AI requests originate from the backend so credentials never reach the frontend.

11.3 Privacy & Logging

No login or PII is required by default; only temporary session identifiers are used, expiring after inactivity (future CRM/forms integration would require a privacy review). Logs record API requests, processing time, errors, sync events, and AI provider status — but API keys, tokens, and confidential configuration values must never be written to logs.

11.4 Security Risk Mitigation

Risk	Mitigation
Prompt injection	Restrict responses to retrieved KB content; enforce system prompt rules
API abuse	Rate limiting and request validation
API key exposure	Server-side environment variables only
Malicious/invalid input	Sanitize and validate all input before processing
Hallucinated responses	RAG with confidence-based validation
Service outage	User-friendly fallback message; log incident for investigation

11.5 Bot Prevention & CAPTCHA (Future Scope)

Planned for a future release, not included in the initial deployment. An invisible CAPTCHA (e.g., reCAPTCHA v3 or Cloudflare Turnstile) protects the AI provider budget from automated abuse: the widget silently generates a `captcha_token`, submitted with each chat request; FastAPI verifies it with the provider before invoking retrieval; a bot-like score returns `403 Forbidden` without ever calling the LLM, incurring zero API cost. As a second layer, IP-based rate limiting (e.g., max 20 messages per 10 minutes) guards against manual or distributed spam even if CAPTCHA is bypassed.

Decision	Justification
HTTPS everywhere	Protects data in transit
Backend-only AI communication	Prevents credential exposure to the client
Invisible CAPTCHA (future)	Protects API budget without harming UX
Temporary session identifiers	Conversational continuity without requiring authentication

12. Deployment Strategy

12.1 Environments & Containers

The chatbot is containerized with Docker for consistent, portable deployment across three environments: **Development** (feature work, local testing), **Testing/UAT** (validation before release), and **Production** (live traffic). Initial hosting is on-premise for full control, predictable cost, and easier compliance, while remaining cloud-ready (AWS/Azure/GCP) with minimal changes if needed.

Container	Responsibility
Nginx	Reverse proxy, SSL termination, routing
FastAPI	REST APIs and chatbot business logic
Qdrant (Lightweight)	Vector storage and semantic search
PostgreSQL + pgvector (Enterprise)	Vector storage, analytics, operational data
pgAdmin (Optional)	PostgreSQL administration

12.2 Deployment Options

OPTION A — LIGHTWEIGHT

FastAPI + Qdrant + Nginx + Docker. Best for minimal resources, fast setup, low maintenance, no conversation storage.

OPTION B — ENTERPRISE

Adds PostgreSQL + pgvector + pgAdmin for conversation analytics, token/cost tracking, feedback analysis, and an operational dashboard.

12.3 Configuration, Backup & Workflow

All configuration — API keys, DB connection strings, LLM provider, ports, logging, session timeout — is externalized via environment variables, never hardcoded. Scheduled backups cover PostgreSQL/Qdrant data, config files, and source code, with periodic restore testing. Since the knowledge base derives from the live website, embeddings can be fully regenerated if lost — it is not a single point of failure.

1. Pull the latest source code.
2. Update configuration files and environment variables.

3. Build Docker images and start with `docker compose up`.
4. Verify database and AI provider connectivity.
5. Run API health checks (`/api/v1/health`) and trigger knowledge sync.
6. Validate chatbot functionality end-to-end and release to production traffic.

13. Monitoring & Logging

13.1 Objectives & Coverage

Monitoring targets three areas — **Application Health**, **AI Performance**, and **Knowledge Base Freshness** — to ensure availability, catch issues early, measure response quality, track cost, and support troubleshooting.

13.2 Metrics

Category	Key Metrics
Application Health	API availability, response time, error rate, active sessions, request volume
AI Performance	Total AI requests, generation time, token consumption, estimated cost, retrieval confidence, provider utilization
Knowledge Base	Total indexed pages/chunks, last sync, failed syncs, knowledge version

13.3 Logging Strategy & Key Decisions

Logs capture API requests, AI requests, synchronization events, warnings, and errors using standard levels (INFO, WARNING, ERROR, CRITICAL) to keep noise low while enabling fast troubleshooting. API keys, tokens, and credentials must never be written to logs.

Decision	Justification
Continuous health monitoring	Ensures reliable availability
Structured logging	Simplifies troubleshooting
Knowledge sync monitoring	Ensures responses stay current

14. AI System Validation & Testing

14.1 Scope & Approach

Because the chatbot uses a RAG architecture, testing extends beyond API checks to retrieval accuracy, prompt effectiveness, chunking quality, and response reliability — combining manual evaluation with technical verification across: content processing, chunking pipeline, embedding generation, vector database retrieval, prompt builder, LLM response quality, backend logic, and end-to-end frontend integration.

Manual retrieval validation: prepare test questions covering all major website sections, submit each through the chatbot, inspect retrieved chunks before generation, verify relevance, and log issues for follow-up. **Chunking** is reviewed for logical boundaries, complete ideas, and minimal duplication — poor chunking degrades quality even when retrieval works. **Prompt changes** trigger full regression testing before deployment.

14.2 Response Accuracy Criteria

Criterion	Description
Accuracy	Response matches website content
Completeness	No important information omitted
Relevance	Directly addresses the question
Consistency	Similar questions get consistent answers
Grounding	Supported by retrieved content

14.3 Regression, Performance & Fallback Testing

Regression testing runs after any prompt modification, content update, sync, provider change, or backend change, using a standardized reference question set spanning every major website category. Performance is measured via average response time, retrieval latency, generation time, and concurrent request handling. Fallback behavior is tested against unrelated questions, incomplete/invalid/empty input, and no-match queries — the expected behavior is to never fabricate an answer, but state that information is unavailable and recommend the relevant page or contact channel.

14.4 Evaluation Scorecard

Each test question is scored across pipeline stages rather than simple pass/fail, so a failure points to exactly where it occurred:

Test ID	Question	Retrieved?	Prompt OK?	Grounded?	Status
TC-001	AI services offered?	✅	✅	✅	Pass
TC-002	Flutter app development?	✅	✅	✅	Pass
TC-003	Who is the CEO of Google?	N/A	✅	✅ (Fallback)	Pass

The chatbot is deployment-ready when content is fully indexed, retrieval consistently returns relevant results, chunk/prompt quality is verified, responses accurately reflect website information, fallback behavior works correctly, performance meets targets, and no critical defects remain. Validation continues after deployment as the knowledge base evolves.

15. Risk Assessment & Mitigation

15.1 Risk Register

Risk	Impact	Probability	Mitigation
Website content becomes outdated	High	Medium	Event-driven incremental synchronization
AI hallucination	High	Medium	RAG grounding restricted to retrieved content
API provider downtime	High	Low	Fallback message, provider monitoring, future multi-provider support
Increased API costs	Medium	Medium	Token monitoring, prompt/retrieval optimization
Low-confidence retrieval	Medium	Medium	Confidence thresholds; redirect to Contact/ Inquiry
Vector database corruption/failure	High	Low	Regular backups, restore validation
Unexpected application failures	High	Low	Structured logging, health monitoring, alerting

15.2 Technical, Operational & Business Risks

Category	Risk	Mitigation
Technical	Retrieved content doesn't closely match the question	Semantic embeddings, chunk tuning, threshold tuning, continuous evaluation
Technical	Changes in external AI API performance/availability	Provider-independent abstraction, timeouts, structured error handling
Operational	Insufficient server resources	Containerized deployment, resource monitoring, flexible hosting upgrades
Operational	Database service disruption	Connection pooling, automated restarts, regular backups
Business	Incorrect answers eroding user trust	RAG grounding, source links, conservative confidence thresholds
Business	Unpredictable API costs at scale	Token tracking, prompt optimization, optional rate limiting

15.3 Risk Review Process

Risks are reviewed periodically during development and after deployment: **1)** analyze API cost, token usage, and latency metrics; **2)** review user feedback (helpful/unhelpful, comments); **3)** audit failed, unanswered, or low-confidence queries; **4)** assess accuracy and hallucination rate; **5)** expand knowledge base coverage to address identified gaps.

16. Future Roadmap

16.1 Phased Evolution

Phase	Objective	Key Deliverables
Phase 1 — MVP	Production-ready RAG chatbot	Website-based retrieval, Gemini integration (OpenAI as tested alternative), local Sentence Transformer embeddings, FastAPI + Docker, PostgreSQL + pgvector, Webflow widget with session memory, source-aware fallback handling
Phase 2 — Monitoring & Analytics	Operational visibility	Admin dashboard, request/response/conversation logging, performance and usage monitoring, knowledge sync auditing, AI-powered conversation summaries, user feedback collection, CAPTCHA and rate limiting
Phase 3 — UX Enhancements	Richer, more accessible interaction	Voice interaction and file uploads, rich response cards, suggested follow-up questions, multi-language support, enhanced conversation memory, continued accessibility improvements
Long-Term Vision	Enterprise-scale evolution	Multi-website support, improved RAG/semantic search accuracy, advanced conversation analytics, automated reporting, algorithmic response-quality evaluation, fully automated knowledge synchronization

The phased roadmap enables incremental development while minimizing implementation risk — each phase builds on the existing foundation, allowing the chatbot to evolve into a scalable, intelligent customer support solution without a complete redesign.

Appendix A – LLM Cost Comparison

A.1 LLM Pricing Reference

Model	Provider	Input (/ 1M)	Output (/ 1M)	Suitability
Gemini 3.1 Flash Lite	Google	$0.25	$1.50	Default choice for high-speed RAG Q&A
GPT-5.4 mini	OpenAI	$0.75	$4.50	Secondary comparison & fallback model
Gemini 3.5 Flash	Google	$1.50	$9.00	Advanced reasoning & multi-turn context
GPT-5.6 Luna	OpenAI	$1.00	$6.00	Enterprise fallback model

A.2 Per-Conversation Cost & Reduction Strategies

Assuming ~2,000 input tokens (retrieved context + prompt + question) and a 500-token response on Gemini 3.1 Flash Lite: input cost ≈ $0.0005, output cost ≈ $0.00075, for a total of **~$0.00125 per conversation**.

> **ROI IMPACT**
>
> At this rate, 1,000 complete customer conversations cost approximately **$1.25 USD** in AI API fees — extremely low operational expense.

Strategy	Impact
Local Embedding Models	100% savings on embedding API fees — Sentence Transformers runs locally at zero per-token cost
Semantic Caching	Caching common answers (hours, location, core services) reduces LLM calls by 30–40%
Self-Hosted Vector Store	Qdrant/PostgreSQL+pgvector in Docker avoids monthly cloud vector-DB subscriptions
Prompt Compression & Chunk Pruning	Filtering low-relevance chunks keeps context under 2,000 tokens, lowering input billing

Appendix B – Requirement Gathering Questions

These discovery questions shape the chatbot's scope, data architecture, and infrastructure decisions before implementation begins.

B.1 Project Scope & Knowledge Base

- **Which website(s) should the chatbot support** — one domain or multiple (e.g., .in, .com, .ca)? Determines whether a single knowledge base suffices or separate collections/metadata are needed.
- **How frequently is website content updated** (service pages, portfolios, case studies, blogs)? Sets the appropriate synchronization schedule.
- **Can the knowledge base update be automated via the existing CMS**, or will updates require manual notification? Determines whether sync can be CMS-triggered or must remain manual.

B.2 Existing Chatbot & Audience

- **Does the existing manual inquiry widget store conversation data** (queries, responses, timestamps), and if so, where and is it accessible? Historical data could reveal common questions and inform the new knowledge base.
- **Who is the primary audience** — potential clients, existing clients, job seekers, or general visitors? Shapes communication style, response depth, and prioritized content.

B.3 Placement, Infrastructure & Administration

- **Should the chatbot appear on every page or only selected pages?** Affects frontend integration and any context-specific behavior needed.
- **Does the existing server have sufficient CPU, RAM, and storage?** Determines whether on-premise hosting is sufficient or cloud deployment is required.
- **Is an admin dashboard required** for knowledge base management, configuration, or analytics? Directly informs the choice between Qdrant and PostgreSQL + pgvector.

B.4 AI Usage & Budget

- **Is there a monthly budget limit for AI API usage?** Influences model selection, response generation strategy, and long-term cost planning.